Code:-

```
roomA=int(input("Enter room A status(0=Clean, 1=Dirty): "))
roomB=int(input("Enter room B status(0=Clean, 1=Dirty): "))
startPos=int(input("Enter starting position of vaccumn(Room A=1, Room b=2):"))

if startPos>2 or startPos<0:
   print("Invalid Input")


while roomA+roomB>0:
   if startPos==1:
      if roomA==0:
         print("Room A is clean. Moving to Room B")
         startPos+=1
      elif roomA==1:
         print("Room A is dirty. Cleaning it.")
         roomA=0
         print("Room A is now clean")
   else:
      if roomB==0:
         print("Room B is clean. Moving to Room A")
         startPos-=1
      elif roomB==1:
         print("Room B is dirty. Cleaning it.")
         roomB=0
         print("Room B is now clean")

print("Both the rooms are now clean")
```

Output:-

```
PS D:\Third Year\SEM-V\AI\AI Lab-I\AI Assignments> python vaccum.py
Enter room A status(0=Clean, 1=Dirty): 0
Enter room B status(0=Clean, 1=Dirty): 1
Enter starting position of vaccumn(Room A=1, Room b=2):1
Room A is clean. Moving to Room B
Room B is dirty. Cleaning it.
Room B is now clean
Both the rooms are now clean
```

Code:-

```
from collections import deque

# ----------------------------------------
# GRID ENVIRONMENT
# ----------------------------------------

grid = [
    ['S', '.', '.', '#', '.'],
    ['#', '#', '.', '#', '.'],
    ['.', '.', '.', '.', '.'],
    ['.', '#', '#', '#', '.'],
    ['.', '.', '.', '.', 'G']
]

ROWS = len(grid)
COLS = len(grid[0])

start = (0, 0)
goal = (4, 4)

# ----------------------------------------
# MODEL-BASED INTELLIGENT AGENT
# ----------------------------------------

class ModelBasedAgent:

    def __init__(self, start, goal):
        self.position = start
        self.goal = goal

        # Internal state representation
        self.internal_map = {}

        # Cells already visited
        self.visited = set()

        # Path followed by the agent
        self.path = [start]

    # ---------------------------------------
    # PERCEIVE ENVIRONMENT
    # ---------------------------------------
    def perceive(self, grid):

        r, c = self.position

        # Observe current position
        self.internal_map[(r, c)] = grid[r][c]

        # Observe neighboring cells
```

```
    directions = {
        "UP": (-1, 0),
        "DOWN": (1, 0),
        "LEFT": (0, -1),
        "RIGHT": (0, 1)
    }

    for action, (dr, dc) in directions.items():

        nr = r + dr
        nc = c + dc

        if 0 <= nr < ROWS and 0 <= nc < COLS:
            self.internal_map[(nr, nc)] = grid[nr][nc]

# --------------------------------------
# CHECK WHETHER CELL IS VALID
# --------------------------------------
def is_valid(self, position):

    r, c = position

    if r < 0 or r >= ROWS:
        return False

    if c < 0 or c >= COLS:
        return False

    # Avoid known obstacles
    if self.internal_map.get(position) == '#':
        return False

    return True

# --------------------------------------
# FIND PATH USING INTERNAL MODEL
# --------------------------------------
def find_path(self):

    queue = deque()

    queue.append((self.position, [self.position]))

    visited = set()

    visited.add(self.position)

    directions = [
        (-1, 0),   # UP
        (1, 0),    # DOWN
        (0, -1),   # LEFT
        (0, 1)     # RIGHT
```

```
    ]

    while queue:

        current, path = queue.popleft()

        if current == self.goal:
            return path

        r, c = current

        for dr, dc in directions:

            next_position = (r + dr, c + dc)

            if (next_position not in visited and
                self.is_valid(next_position)):

                visited.add(next_position)

                new_path = path + [next_position]

                queue.append(
                    (next_position, new_path)
                )

    return None

# -------------------------------------
# SELECT ACTION
# -------------------------------------
def select_action(self, next_position):

    r, c = self.position
    nr, nc = next_position

    if nr == r - 1:
        return "UP"

    elif nr == r + 1:
        return "DOWN"

    elif nc == c - 1:
        return "LEFT"

    elif nc == c + 1:
        return "RIGHT"

    return "STOP"

# -------------------------------------
# MOVE AGENT
```

```
# ------------------------------------
def move(self, next_position):

    action = self.select_action(next_position)

    print("Current Position :", self.position)
    print("Action           :", action)

    self.position = next_position

    self.visited.add(self.position)
    self.path.append(self.position)

    print("New Position     :", self.position)
    print("--------------------------------")

# ------------------------------------
# RUN AGENT
# ------------------------------------
def run(self, grid):

    print("\n===== MODEL-BASED AGENT =====\n")

    self.visited.add(self.position)

    while self.position != self.goal:

        # Step 1: Perceive environment
        self.perceive(grid)

        # Step 2: Update internal model
        print("Internal State:")
        print(self.internal_map)

        # Step 3: Find path using model
        path = self.find_path()

        if path is None:
            print("No path to goal found.")
            return

        # Get next position
        if len(path) > 1:
            next_position = path[1]

            # Step 4: Select action and move
            self.move(next_position)

    print("\nGoal Reached!")
    print("Goal Position :", self.goal)
    print("Complete Path :", self.path)
    print("Visited Cells :", self.visited)
```

```
# ----------------------------------------
# DISPLAY GRID
# ----------------------------------------

def display_grid(grid):

    print("\nGrid Environment:\n")

    for row in grid:
        print(" ".join(row))

# ----------------------------------------
# MAIN PROGRAM
# ----------------------------------------

display_grid(grid)

agent = ModelBasedAgent(start, goal)

agent.run(grid)
```

Output:-

```
PS D:\Third Year\SEM-V\AI\AI Lab-I\AI Assignments> python temp.py

Grid Environment:

S . . # .
# # . # .
. . . . .
. # # # .
. . . . G

===== MODEL-BASED AGENT =====

Internal State:
{(0, 0): 'S', (1, 0): '#', (0, 1): '.'}
Current Position : (0, 0)
Action           : RIGHT
New Position     : (0, 1)
---------------------------------
Internal State:
{(0, 0): 'S', (1, 0): '#', (0, 1): '.', (1, 1): '#', (0, 2): '.'}
Current Position : (0, 1)
Action           : RIGHT
New Position     : (0, 2)
---------------------------------
Internal State:
{(0, 0): 'S', (1, 0): '#', (0, 1): '.', (1, 1): '#', (0, 2): '.', (1, 2): '.', (0, 3): '#'}
Current Position : (0, 2)
```

```
Action          : DOWN
New Position    : (1, 2)
--------------------------------
Internal State:
{(0, 0): 'S', (1, 0): '#', (0, 1): '.', (1, 1): '#', (0, 2): '.', (1, 2): '.', (0, 3): '#', (2, 2): '.', (1, 3): '#'}
Current Position : (1, 2)
Action          : DOWN
New Position    : (2, 2)
--------------------------------
Internal State:
{(0, 0): 'S', (1, 0): '#', (0, 1): '.', (1, 1): '#', (0, 2): '.', (1, 2): '.', (0, 3): '#', (2, 2): '.', (1, 3): '#', (3, 2): '#', (2,
1): '.', (2, 3): '.'}
Current Position : (2, 2)
Action          : RIGHT
New Position    : (2, 3)
--------------------------------
Internal State:
{(0, 0): 'S', (1, 0): '#', (0, 1): '.', (1, 1): '#', (0, 2): '.', (1, 2): '.', (0, 3): '#', (2, 2): '.', (1, 3): '#', (3, 2): '#', (2,
1): '.', (2, 3): '.', (3, 3): '#', (2, 4): '.'}
Current Position : (2, 3)
Action          : RIGHT
New Position    : (2, 4)
--------------------------------
Internal State:
{(0, 0): 'S', (1, 0): '#', (0, 1): '.', (1, 1): '#', (0, 2): '.', (1, 2): '.', (0, 3): '#', (2, 2): '.', (1, 3): '#', (3, 2): '#', (2,
1): '.', (2, 3): '.', (3, 3): '#', (2, 4): '.', (1, 4): '.', (3, 4): '.'}
Current Position : (2, 4)
Action          : DOWN
New Position    : (3, 4)
--------------------------------
Internal State:
{(0, 0): 'S', (1, 0): '#', (0, 1): '.', (1, 1): '#', (0, 2): '.', (1, 2): '.', (0, 3): '#', (2, 2): '.', (1, 3): '#', (3, 2): '#', (2,
1): '.', (2, 3): '.', (3, 3): '#', (2, 4): '.', (1, 4): '.', (3, 4): '.', (4, 4): 'G'}
Current Position : (3, 4)
Action          : DOWN
New Position    : (4, 4)
--------------------------------

Goal Reached!
Goal Position : (4, 4)
Complete Path : [(0, 0), (0, 1), (0, 2), (1, 2), (2, 2), (2, 3), (2, 4), (3, 4), (4, 4)]
Visited Cells : {(0, 1), (4, 4), (2, 4), (1, 2), (3, 4), (0, 0), (2, 3), (0, 2), (2, 2)}
PS D:\Third Year\SEM-V\AI\AI Lab-I\AI Assignments>
```